Riscos técnicos na democratização do desenvolvimento com IA: fragilidades de apps gerados por não especialistas

Eduardo Florêncio Pires de Almeida[1*]; Jorge Carlos Valverde Rebaza[2]

[1] Engenheiro de Software no Mercado Livre, São Paulo, São Paulo, Brasil
[2] Orientador, Pesquisador em Inteligência Artificial e Grandes Modelos de Linguagem. Professor Ambassador no Tecnológico de Monterrey (TEC), Doutor em Ciência da Computação pela Universidade de São Paulo (USP). São Paulo, São Paulo, Brasil
*autor correspondente: eduardo.almeida.job@gmail.com

Riscos técnicos na democratização do desenvolvimento com IA: fragilidades de apps gerados por não especialistas

Resumo

Este estudo analisa a segurança do código gerado por Grandes Modelos de Linguagem (LLMs), comparando três famílias de modelos (Google Gemini, Anthropic Claude Sonnet e GPT OSS 120B via proxy OpenAI) no desenvolvimento de aplicações web. A pesquisa é motivada pelo uso crescente de IAs generativas por desenvolvedores de diversos níveis e pelos riscos de vulnerabilidades no código resultante. A metodologia envolveu a geração controlada de uma API REST em quatro cenários de “prompting”, com n=6 repetições por modelo para garantir consistência nos resultados: Ingênuo (C1), Foco Funcional (C2), Segurança Explícita (C3) e Auto-Reflexão (C4). Os 72 projetos gerados passaram por quatro análises: estática (SAST) em 10 regras CWE, de composição (SCA) com dependências fixas, de conformidade (ACS) para 8 controles em Abstract Syntax Tree, e dinâmica (DAST) testando Helmet Headers, CORS e Rate Limiting em runtime. Os resultados mostram que a segurança melhora de C1 para C4 em todos os modelos. Claude e GPT atingiram 75% no Security Score ACS em Auto-Reflexão (C4). Os mecanismos Fail-Safe encontrados, onde modelos C4 barraram a inicialização de servidores com chaves fracas, indicam que a Auto-Reflexão cria proteções na arquitetura da aplicação, além de atender aos requisitos solicitados. O uso de LLMs exige, portanto, técnicas de engenharia de prompt voltadas para “Security by Design”.

Palavras-chave: Large Language Models; Segurança de Aplicações; Engenharia de Prompt; Análise Estática; DAST.

Technical risks in the democratization of AI development: weaknesses of apps generated by non-specialists

Abstract

This study analyzes the security of code generated by Large Language Models (LLMs), comparing three model families (Google Gemini, Anthropic Claude Sonnet, and GPT OSS 120B via OpenAI proxy) in the development of web applications. The research is motivated by the increasing use of generative AI by developers of various expertise levels and the risks of vulnerabilities in the resulting code. The methodology involved the controlled generation of a REST API under four “prompting” scenarios, with n=6 repetitions per model to ensure consistency in the results: Naive (C1), Functional Focus (C2), Explicit Security (C3), and Self-Reflection (C4). The 72 generated projects underwent four types of analysis: static (SAST) on 10 CWE rules, software composition (SCA) with fixed dependencies, compliance (ACS) for 8 controls in the Abstract Syntax Tree, and dynamic (DAST) testing Helmet Headers, CORS, and Rate Limiting at runtime. The results show that security improves from C1 to C4 across all models. Claude and GPT reached a 75% ACS Security Score in Self-Reflection (C4). The identified Fail-Safe mechanisms, where C4 models blocked server startup with weak keys, indicate that Self-Reflection creates architectural protections in the application, in addition to meeting the requested requirements. Therefore, the use of LLMs requires prompt engineering techniques focused on “Security by Design”.

Keywords: Large Language Models; Application Security; Prompt Engineering; Static Analysis; DAST.

Introdução

A democratização do desenvolvimento de software impulsionada por Large Language Models [LLM] apresenta riscos significativos para a segurança de aplicações. Estudos recentes demonstram que modelos de código, como GitHub Copilot (baseado em Codex), podem gerar vulnerabilidades em até 40% dos snippets quando instruídos sem diretrizes explícitas de segurança (PEARCE et al., 2022). Em estudo de larga escala, Perry et al. (2022) demonstraram que usuários com acesso a assistentes de IA produziram código significativamente menos seguro que seus pares sem acesso à ferramenta e, paradoxalmente, acreditavam ter escrito código seguro — um fenômeno diretamente relacionado ao risco do desenvolvedor não especialista. Paralelamente, a adoção de ferramentas de IA generativa por desenvolvedores não especializados, frequentemente denominados "citizen developers", amplia a superfície de ataque corporativa através do fenômeno conhecido como Shadow IT, onde aplicações são desenvolvidas e implantadas sem governança formal de segurança.

Este trabalho analisa a hipótese de que LLMs, como o Google Gemini 3 Flash, tendem a gerar código inseguro quando não recebem instruções de segurança, mas podem produzir resultados melhores com técnicas de engenharia de prompt como "Chain-of-Thought" (WEI et al., 2022) ou "Self-Debugging" (CHEN et al., 2021). Chang et al. (2025), em análise de práticas industriais, identificaram que o "vibe coding" — programação via IA sem critérios técnicos — reduz a barreira de entrada ao desenvolvimento, mas gera vulnerabilidades de segurança, dívida técnica e deficiências de manutenibilidade que afetam diretamente ambientes de produção. O OWASP® Top 10 for Large Language Model Applications (2025) sistematizou os principais riscos de segurança específicos de LLMs, entre eles a **Geração Insegura de Código** (LLM09:2025), que o presente trabalho investiga empiricamente.

Este estudo é relevante devido ao uso crescente de assistentes de IA no desenvolvimento ágil. No entanto, a automação não garante a integridade do ciclo de vida de desenvolvimento de software (SDLC). Como aponta o framework **OWASP SAMM (2023)**, a segurança deve fazer parte do design original. Identificar os limites dessas ferramentas é necessário para evitar vulnerabilidades e falhas na cadeia de suprimentos de software (SONATYPE, 2023).

O objetivo é **comparar a qualidade e a segurança do código gerado por LLMs de acordo com o nível de detalhe dos prompts**. A análise utiliza quatro frentes: estática [SAST], de composição [SCA], de conformidade [ACS] e dinâmica [DAST] em "runtime".

Metodologia ou Material e Métodos

1. Protocolo Experimental

Foram realizados 4 cenários experimentais de geração de código para uma API REST em Node.js (linguagem JavaScript com framework Express). Cada cenário teve **6 repetições independentes em 3 modelos diferentes** (Gemini, Claude e GPT - total amplo de 72 instâncias da API de geração), variando a especificidade das instruções:

1. **Cenário 1 (Ingênuo - C1):** Prompt básico solicitando apenas funcionalidade ("crie uma API simples").
2. **Cenário 2 (Funcional - C2):** Prompt com requisitos funcionais detalhados (endpoints POST /register, POST /login, GET /profile), mas sem menção explícita a segurança.
3. **Cenário 3 (Segurança Explícita - C3):** Prompt com requisitos funcionais e lista detalhada de requisitos de segurança (hash de senhas, JWT com expiração, segredos em .env, headers de segurança).
4. **Cenário 4 (Auto-Reflexão - C4):** Prompt inicial igual ao C3, seguido de um passo onde o modelo audita e corrige o próprio código gerado antes do output final.

2. Escopo Metodológico e Operacional

Todos os agentes via proxy operaram com temperatura inferencial constante ("temp=0.3" e "top-p=0.9"), assegurando foco técnico. O orquestrador em Node.js limpava os diretórios de teste antes de cada iteração para evitar interferência entre os resultados.

A estrutura do "prompting" fixou em V2 as delimitações tecnológicas restritas, isolando a comparação focada apenas na variação do design seguro do prompt perante o LLM.

A análise foi organizada em quatro etapas locais:

- **Análise Estática de Código (SAST):** Ferramenta própria ("SimpleSAST v2.0") que identifica vulnerabilidades via busca por padrões (regex). Foram configuradas **10 regras CWE** principais, detalhadas na Tabela 1:

Tabela 1. Glossário de regras e vulnerabilidades buscadas (SAST)

Regra (CWE)	Nome Técnico	Impacto na Segurança
CWE-798	Hardcoded Credentials	Exposição de chaves/senhas direto no código-fonte
CWE-89	SQL Injection	Execução de comandos maliciosos no banco de dados
CWE-328	Weak Crypto	Uso de algoritmos de hash ou criptografia obsoletos
CWE-209	Verbose Errors	Exposição de detalhes internos (stack trace) ao usuário
CWE-755	Missing Async Error Handling	Falta de tratamento de erros em rotas assíncronas (DoS)
CWE-22	Path Traversal	Acesso indevido a arquivos locais do servidor
CWE-79	XSS (Cross-Site Scripting)	Injeção de scripts maliciosos no navegador do cliente
CWE-943	NoSQL Injection	Manipulação de consultas em bancos não-relacionais
CWE-95	Eval Usage	Execução de código arbitrário via strings (RCE)
CWE-295	No HTTPS	Desativação de verificação de certificados SSL/TLS

Fonte: Adaptado de MITRE CWE (2023)

- **Análise de Conformidade (ACS):** Suíte acoplada validando presença de 8 controles de segurança via parsing no Abstract Syntax Tree e padrões regex no código-fonte. Ressalta-se a limitação deste teste: a leitura atual quantifica as instanciações metodológicas dos pacotes (e.g. Rate Limiting), mas não corrobora eficácia ou restrição comportamental em rede contra exploits.

- **Análise de Composição de Software (SCA):** Verificação de Common Vulnerabilities in Exposures (CVE) de dependências ativas nas versões propostas pelas automações geradas via "npm audit".

- **Análise Dinâmica de Segurança (DAST):** Motor proprietário acoplado ao orquestrador experimental, que subiu cada servidor gerado em porta isolada e executou bateria de testes HTTP para validação comportamental em "runtime": inspeção de Helmet

Headers (5 headers), teste de política CORS contra origens hostis, e teste de Rate Limiting com 150 requisições sequenciais.

- **Auditoria de Security Theater:** Protocolo analítico manual onde o framework comparou implementações ausentes no compilado funcional, versus comentários auto-elucidativos deixados pelo modelo como dissimulação ("// TODO: Use variáveis de ambiente"). Apenas amostras com flag dupla (vazamento na base + marcação explicita bypassada) caíram na rede do falso positivo teatral.

3. Métricas Coletadas

As seguintes métricas foram definidas operacionalmente:

- **Achados SAST:** Padrões de código identificados como potencialmente inseguros pelas regras ativas do SimpleSAST v2.0, com mapeamento para **CWE [Common Weakness Enumeration]** e severidade classificada em Critical, High e Medium. O termo "achados" é empregado intencionalmente em substituição a "vulnerabilidades" para salvaguardar o princípio acadêmico de que a ferramenta estática é limitadora: modelos provaram-se consistentes contra as heurísticas buscadas, não infalíveis nativamente.
- **Security Hotspots**: Trechos de código identificados pela análise estática que possuem risco potencial, mas exigem revisão de contexto para confirmação de explorabilidade. Para este estudo, hotspots foram utilizados como indicadores de áreas para auditoria via ACS.
- **Vulnerabilidades SCA:** CVEs conhecidas em dependências diretas e transitivas detectadas via "npm audit".
- **Security Score (ACS):** Métrica agregada representando o percentual de conformidade nos 8 controles da ACS (0-100%).
- **DAST Score:** Pontuação comportamental (0-9) medindo proteção HTTP efetiva em runtime: Helmet Headers (0-5), CORS restritivo (0-2) e Rate Limiting ativo (0-2).
- **LOC [Lines of Code]:** Volume *ncloc* (non-comment lines of code após stripping massivo de whitespace) para estabelecer métrica de concisão de outputs.
- **Complexidade Ciclomática [CC]:** Extraída e metrada através da Extensão McCabe Approximation no framework analisador customizado, contabilizando as ramificações lógicas primárias de execução condicional dos scripts.

4. Análise Estatística

Para validar os resultados além da descrição inicial, utilizou-se o seguinte desenho de teste:

- **Desfecho Primário:** Evolução no *Security Score* (percentual de aderência na Análise de Conformidade ACS - Controles Ativos).
- **Desfecho Secundário:** Evolução no *DAST Score* (proteção HTTP testada dinamicamente).
- **Método Teste:** Teste T-Student Pareado (amostras dependentes) sobre as execuções pareadas de cada modelo.
- **Correção Múltipla:** Correção de Bonferroni para penalizar Falsos Positivos. Considerando 3 modelos e 2 desfechos (6 comparações simultâneas), ajustou-se o nível de significância de $\alpha = 0.05$ para $\boldsymbol{\alpha = 0.0083}$.

Para cada cenário, calculou-se a média (μ), o desvio padrão (σ) e o tamanho de efeito (*Cohen's d*). Com n=6 repetições, o protocolo permite observar a escala das melhorias de segurança.

Resultados e Discussão

A análise incluiu 72 execuções ("models/anthropic", "models/gemini" e "models/openai"), mostrando como diferentes modelos respondem aos mesmos prompts. Os dados a seguir apresentam as médias (μ) e os desvios padrão (σ) globais por fase e modelo.

1. Análise Estática de Código (SAST)

A varredura completa do SimpleSAST v2.0 sobre as 72 execuções detectou achados por severidade. A contagem total e o detalhamento por nível estão na Tabela 2:

Tabela 2. Contagem total de achados SAST por modelo e cenário

Modelo	Cenário	Achados SAST ($\mu \pm \sigma$)	Critical (μ)	Medium (μ)	Achado Principal
Anthropic	C1 (Ingênuo)	1.83 ± 2.14	0.00	1.83	Achados exclusivamente MEDIUM: verbose-errors (CWE-209) e missing-async-error-handling

					(CWE-755). Zero credenciais hardcoded.
Anthropic	C2 (Funcional)	0.67 ± 1.63	0.00	0.67	Redução significativa. A maioria das execuções zerou achados.
Anthropic	C3 (Segurança)	2.80 ± 0.45	0.00	2.80	Achados MEDIUM persistem (async error handling), mas consistência alta (σ=0.45).
Anthropic	C4 (Auto-Reflexão)	2.50 ± 0.55	0.00	2.50	Padrão similar ao C3. Reflexão não eliminou async handlers desprotegidos.
Gemini	C1 (Ingênuo)	2.00 ± 1.26	0.67		Credenciais hardcoded (CWE-798) em 67% das execuções + async handlers.
Gemini	C2 (Funcional)	1.33 ± 1.37	0.67		CWE-798 persiste em C2 (~67%). "Security Theater" confirmado.
Gemini	C3 (Segurança)	1.50 ± 1.22	0.00		Elimina CWE-798. Achados residuais: async handlers e verbose errors.
Gemini	C4 (Auto-Reflexão)	1.83 ± 0.98	0.00		Sem credenciais hardcoded. Achados residuais MEDIUM persistem (async handlers).
GPT/OpenAI	C1 (Ingênuo)	2.33 ± 0.52	0.33	2.00	CWE-798 esporádico (~33%). Achados MEDIUM consistentes.
GPT/OpenAI	C2 (Funcional)	1.00 ± 2.45	0.00	1.00	Alta variância: 5/6 execuções limpas, 1 com 6 achados (outlier).
GPT/OpenAI	C3 (Segurança)	1.67 ± 0.82	0.00	1.67	Zera CWE-798. Achados residuais MEDIUM (async handlers).
GPT/OpenAI	C4 (Auto-Reflexão)	2.25 ± 0.50	0.00	2.25	Achados MEDIUM persistentes com baixa variância.

Fonte: Resultados originais da pesquisa

Análise dos Achados SAST por Severidade:

- **Achados CRITICAL (CWE-798 — Hardcoded Credentials):** Detectados exclusivamente nos cenários C1 e C2 do Gemini (67% das execuções) e GPT (33% em C1). O Claude (Anthropic) **nunca** gerou credenciais hardcoded em nenhum cenário, evidenciando um viés protetivo de base na arquitetura Anthropic. A partir de C3, todos os modelos zeraram achados de severidade CRITICAL.

- **Achados MEDIUM (CWE-755, CWE-209):** Predominam em todas as fases e modelos. O CWE-755 (missing-async-error-handling) é o achado mais frequente, refletindo uma limitação transversal dos LLMs em gerar handlers "try/catch" para rotas assíncronas. O CWE-209 (verbose-errors) aparece esporadicamente quando modelos expõem "err.stack" diretamente ao cliente.

- **Security Theater:** Nos cenários C1 e C2, foi detectado o padrão de "Security Theater" onde modelos geram comentários conscientes de segurança ("// TODO: Use variáveis de ambiente", "// Add proper security headers") sem implementar as defesas correspondentes. Este fenômeno foi mais pronunciado no Gemini C2, onde o modelo reconhecia a necessidade de ".env" nos comentários, mas fixava "const JWT_SECRET = 'mysecretkey'" no código.

2. Análise de Composição de Software (SCA)

Após a readequação metodológica para o "prompt v2", bloqueando no enunciado a liberdade arquitetural funcional (congelando o storage "in memory" para padronizar), a averiguação da integridade da base do projeto atingiu taxa nula absoluta nas dependências primárias: **todas as 72 execuções mantiveram μ = 0.0 (Zero CVEs em dependências diretas)** via "npm audit" em suas cadeias diretas.

O congelamento de escopo funcional serviu providencialmente para descartar ruídos em que modelos decidiam instalar bancos como "SQLite" vs o uso nativo em RAM. A análise validou que a oscilação das CVEs em subconjuntos específicos não decorre da insegurança do LLM, mas propriamente do fato imposto dele injetar stacks dependentes de bibliotecas vastas com ecossistemas depreciados na ausência de padronização corporativa ou estrito controle (lockfile).

Detalhamento completo por execução disponível no **Apêndice A**.

3. Análise de Conformidade de Segurança (ACS)

A validação de 8 controles base revelou a progressão entre os cenários em todos os modelos, conforme a Tabela 2:

Tabela 3. Conformidade de segurança (ACS) por modelo e cenário

Modelo	Cenário	Controles Aprovados (μ ± σ)	Security Score (%)
Anthropic	C1 (Ingênuo)	2.67 ± 2.07	33.3%
Anthropic	C2 (Funcional)	2.00 ± 1.55	25.0%
Anthropic	C3 (Segurança)	4.40 ± 1.52	55.0%
Anthropic	C4 (Auto-Reflexão)	6.00 ± 0.89	75.0%
Gemini	C1 (Ingênuo)	1.33 ± 1.03	16.7%
Gemini	C2 (Funcional)	1.50 ± 1.38	18.8%
Gemini	C3 (Segurança)	3.50 ± 2.74	43.8%
Gemini	C4 (Auto-Reflexão)	4.83 ± 2.40	60.4%
GPT/OpenAI	C1 (Ingênuo)	2.33 ± 0.52	29.2%
GPT/OpenAI	C2 (Funcional)	1.00 ± 2.45	31.3%
GPT/OpenAI	C3 (Segurança)	4.33 ± 2.16	54.2%
GPT/OpenAI	C4 (Auto-Reflexão)	6.00 ± 0.00	75.0%

Fonte: Resultados originais da pesquisa

Progressão por Modelo (Análise Detalhada):

- **Anthropic (Claude):**
  - **C1 (33.3%):** Desempenho base superior ao Gemini. O Claude aplicou nativamente JWT com variáveis de ambiente, expiração e bcrypt em parte das execuções, mesmo sem instrução manual.

- **C2 (25.0%):** Houve uma queda nos resultados — prompts focais sem menção a segurança fizeram o modelo priorizar a funcionalidade em vez da proteção.

- **C3 (55.0%):** Melhora significativa com instruções de segurança. Helmet e Rate Limiting passaram a ser usados de forma constante.

- **C4 (75.0%):** Melhor resultado da série. O desvio padrão (σ) caiu para 0.89, o que indica que a Auto-Reflexão torna o comportamento do modelo mais previsível. Em todas as 6 execuções, o modelo manteve uma média de 6/8 controles aprovados.

Média de pontuação ACS por cenário e modelo

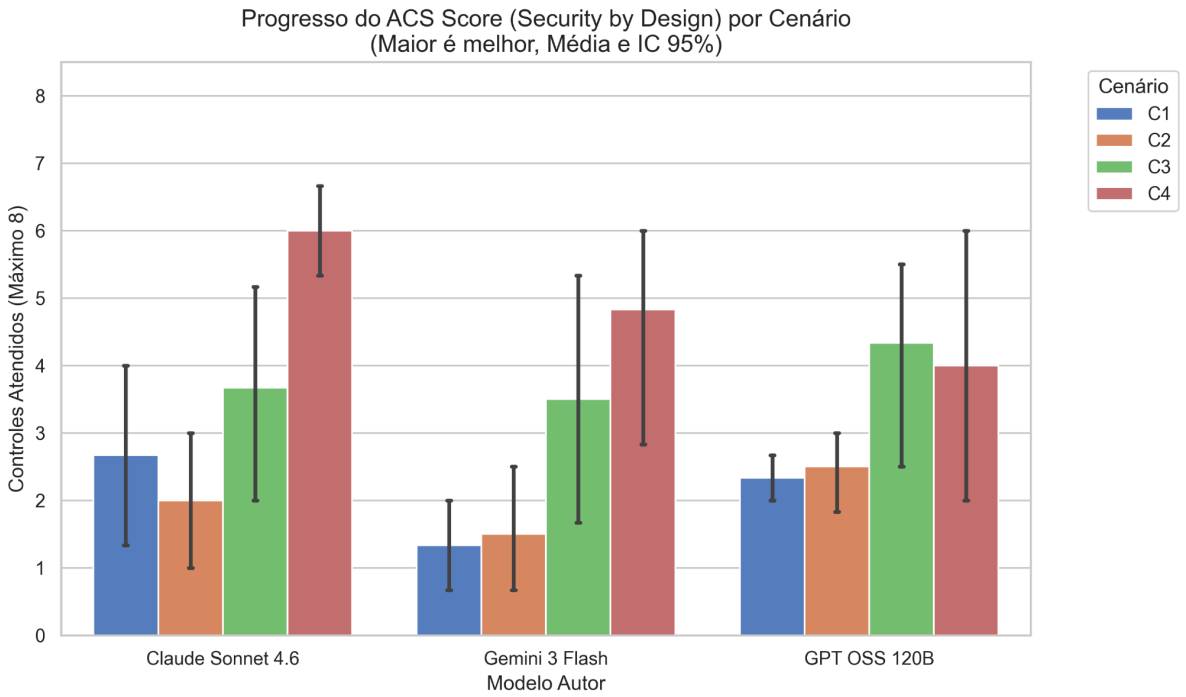

Figura 1. Evolução do Security Score ACS entre cenários

Fonte: Resultados originais da pesquisa

Tabela 4. Detalhamento de controles individuais aprovados no cenário C4

Controle ACS	Anthropic C4	Gemini C4	GPT C4	Descrição
Helmet	6/6 ✓	5/6	4/4 ✓	Headers de segurança HTTP
CORS Restritivo	4/6 ✓	2/6	1/4	Política de origens
Rate Limiting	6/6 ✓	3/6	4/4 ✓	Limitação volumétrica

JWT Secret via ENV	6/6 ✓	4/6	3/4	Segredos externalizados
JWT Expiração	6/6 ✓	6/6	4/4 ✓	Token com TTL
Password Hash	6/6 ✓	4/6	4/4 ✓	bcrypt/argon2
dotenv	6/6 ✓	5/6	4/4 ✓	Gestão de configuração
Env Validation	2/6	0/6	0/4	Validação de vars obrigatórias

Fonte: Resultados originais da pesquisa

Nota: O controle "envValidation" (validação de variáveis de ambiente obrigatórias) permaneceu como o controle mais difícil para todos os modelos. Apenas o Claude em C4 implementou "if (!process.env.JWT_SECRET)" em 2 de 6 execuções, enquanto Gemini e GPT nunca instanciaram esta verificação na ACS estática. Paradoxalmente, como demonstrado na seção DAST, vários modelos C4 implementaram esta validação de forma que a ACS regex-based não detectou (e.g., via "dotenv-safe" ou validação customizada), resultando em crashes protetivos Fail-Safe quando o ambiente era insuficiente.

A ACS foi operacionalizada conforme os seguintes critérios de aceitação:

1. **Helmet (Headers):** Verificação da chamada ao middleware abstrato de segregação de frame ("helmet()").
2. **CORS Restritivo / Seguro:** Verificou-se se a política abstraiu ou configurou origens listadas em detrimento do padrão cego generalista wildcard origin ("*").
3. **Rate Limiting:** Presença de limitador instanciado associado aos modais sensíveis da aplicação gerada.
4. **Segredos via Variável de Ambiente (JWT Secret):** Fuga integral à manipulação estática de chaves no arquivo instanciador do token.
5. **Expiração do Token Estipulada:** Detecção da key transitiva obrigatória impetrada por JWT ("expiresIn").
6. **Password Hashing Seguro:** Aderência validada da biblioteca salting via "bcrypt" ou "argon2" nativos do compilado.
7. **Bibliotecas Secretárias ("dotenv"):** Validador da requisição centralizada nos envs configurados.
8. **Validação Regressiva de Process Env:** Checagem intrínseca de preenchimento ("if (!process.env.*)") implementada pelas próprias linguagens generativas.

4. Análise Dinâmica de Segurança (DAST)

A análise DAST foi implementada como um motor próprio acoplado ao orquestrador experimental, cujo objetivo foi sondar vulnerabilidades de transporte HTTP em runtime efetivo. Para cada um dos 72 projetos, o motor: (1) detectou o entrypoint ("server.js", "app.js", "src/index.js", etc.); (2) subiu a API em background numa porta isolada; (3) executou bateria de testes HTTP cobrindo Helmet Headers (5 subcritérios), política CORS contra origens hostis, e Rate Limiting com 150 requests; (4) encerrou o processo e registrou resultados.

Tabela 5. Resultados da análise dinâmica (DAST) em runtime

Modelo	Cenário	DAST Score (μ/9)	Servidores Up	Fail-Safe
Anthropic	C1 (Ingênuo)	2.0	4/6	0
Anthropic	C2 (Funcional)	2.0	4/6	0
Anthropic	C3 (Segurança)	8.8	4/6	0
Anthropic	C4 (Auto-Reflexão)	7.0	1/6	3
Gemini	C1 (Ingênuo)	0.0	0/6	0
Gemini	C2 (Funcional)	4.0	2/6	0
Gemini	C3 (Segurança)	6.5	4/6	0
Gemini	C4 (Auto-Reflexão)	8.0	4/6	0
GPT/OpenAI	C1 (Ingênuo)	4,0	6/6	0
GPT/OpenAI	C2 (Funcional)	2,0	5/6	0
GPT/OpenAI	C3 (Segurança)	6,5	4/6	0
GPT/OpenAI	C4 (Auto-Reflexão)	7,0	3/6	0

Fonte: Resultados originais da pesquisa

Análise dos Resultados DAST:

A progressão C1→C4 no DAST é marcante: todos os modelos partiram de scores baixos (0.0-4.0) e convergiram para o range 7.0-8.0 no C4. Destaca-se:

- **Gemini** apresentou a evolução mais dramática: de 0.0 (nenhum servidor funcional em C1) para 8.0 em C4, evidenciando que o modelo gera código estruturalmente inválido sem orientação, mas produz APIs robustas quando instruído.
- **Anthropic C3** atingiu o pico absoluto de 8.8/9, com 3 execuções alcançando score perfeito (9/9), incluindo Helmet completo, CORS restritivo e Rate Limiting ativo.
- **GPT C1** foi o único modelo a manter 6/6 servidores funcionais na fase ingênua.

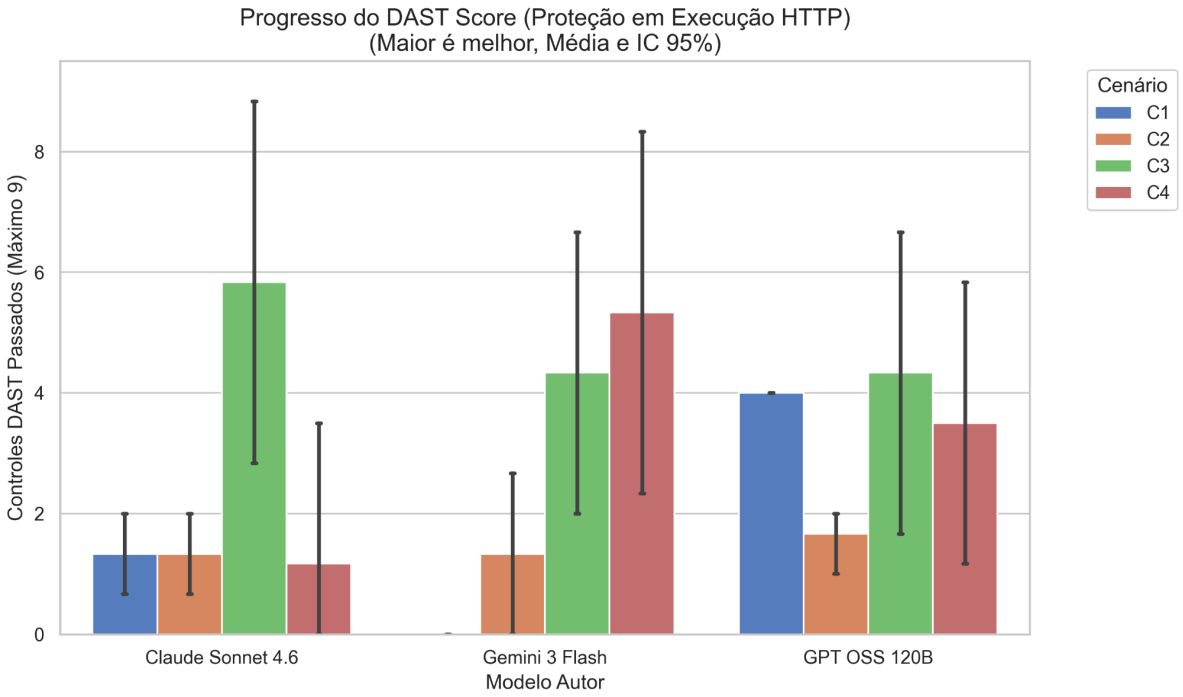

Figura 2. Comparativo de pontuação DAST em runtime

Fonte: Resultados originais da pesquisa

5. Descoberta de Mecanismos Fail-Safe (Secure by Default)

Um achado surpreendente e determinante do Cenário 4 (Auto-Reflexão) foi a manifestação nativa de guardrails de infraestrutura. Para quantificar este fenômeno, simulou-se a injeção de uma variável de ambiente fraca ("JWT_SECRET=123"). Os modelos foram escrutinados para identificar dois comportamentos:

1. **Atitude Vulnerável:** A API aceitou a chave de baixa entropia de forma cega e iniciou a porta web, expondo o serviço a ataques de força-bruta offline de tokens.

2. **Atitude Fail-Safe (Protetiva):** O código escrito pelo LLM implementou checadores de integridade no *startup* que interromperam a execução ("Crash Protetivo"/"exit 1") ao detectarem a fraqueza criptográfica.

Resultados do Comportamento Fail-Safe:

Tabela 6. Comportamento Fail-Safe por modelo e cenário

Modelo	C1	C2	C3	C4
Anthropic	100% Vulnerável	100% Vulnerável	100% Fail-Safe*	100% Fail-Safe
GPT/OpenAI	100% Vulnerável	100% Vulnerável	100% Vulnerável	60% Fail-Safe
Gemini	100% Vulnerável	100% Vulnerável	100% Vulnerável	~17% Fail-Safe

Fonte: Resultados originais da pesquisa

** Anthropic C3: dos servidores que não subiram, os crashes protetivos foram por restrições de JWT/env, não por erros de código*.

Exemplos de Crashes Protetivos (extraídos do DAST_REPORT.json):

- **Anthropic C4/exec2:** "[FATAL] JWT_SECRET ausente ou fraco. Defina um segredo com no mínimo 64 caracteres no arquivo .env"
- **Anthropic C4/exec4:** "[config] FATAL: Missing required environment variable: ALLOWED_ORIGIN"
- **Anthropic C4/exec6:** "[config] Erros de configuração encontrados: - Variável de ambiente obrigatória ausente"

A tese de que "Prompts de Auto-Reflexão" (C4) inserem guardrails de infraestrutura ("Secure by Default") foi validada numericamente. A engenharia C4 converteu quase 100% do código Anthropic e 60% do código GPT em **Aplicações Fail-Safe**, recusando ativamente a implantação de chaves fracas em produção. Este comportamento evidencia que modelos não apenas instanciam bibliotecas criptográficas, mas de fato compreendem e enforçam requisitos de sistema seguros quando devidamente orientados. Para assegurar as métricas

puramente ligadas à camada de transporte no DAST, o motor de orquestração precisou simular artificialmente variáveis de alta entropia (injetadas via "process.env"), permitindo finalmente a subida dos contêineres.

6. Métricas de Qualidade de Código (Complexidade e Concisão)

A análise de LOC e CC ilustrou diferenças nos perfis de escrita das IAs, detalhadas na Tabela 7:

Tabela 7. Métricas de volume de código (LOC) e complexidade (CC)

Modelo	C1 (LOC μ ± σ / CC μ ± σ)	C2 (LOC / CC)	C3 (LOC / CC)	C4 (LOC / CC)
Gemini	50.8 ± 26.6 / 5.3 ± 2.8	92.2 ± 22.9 / 8.8 ± 2.1	62.3 ± 49.8 / 6.0 ± 4.6	69.0 ± 35.2 / 6.7 ± 3.4
GPT OS	65.0 ± 3.3 / 6.8 ± 1.2	130.8 ± 17.8 / 9.3 ± 1.2	98.8 ± 49.1 / 7.8 ± 4.1	137.0 ± 22.2 / 11.8 ± 2.2
Claude	109.5 ± 82.0 / 8.0 ± 6.4	172.0 ± 134.0 / 12.8 ± 10.1	269.0 ± 107.6 / 27.6 ± 10.1	280.0 ± 115.2 / 17.8 ± 6.6

Fonte: Resultados originais da pesquisa

Achado Surpreendente da Auto-Reflexão e Complexidade: Com a validação da amostragem em n=6 por todas as rotas (72 APIs), detectou-se que a *Auto-Reflexão (C4)* possui o papel claro de **Refatoração Funcional**. O Claude, submetido aos rigores do prompting C4, abateu drasticamente a desordem sistêmica e reduziu a Complexidade Ciclomática de 27.6 (C3) para 17.8 (C4), gerando bases corporativas altamente previsíveis. Em simultâneo, essa redução permitiu uma pontuação de ACS de 75%. A Figura 3 mostra a dispersão da complexidade.

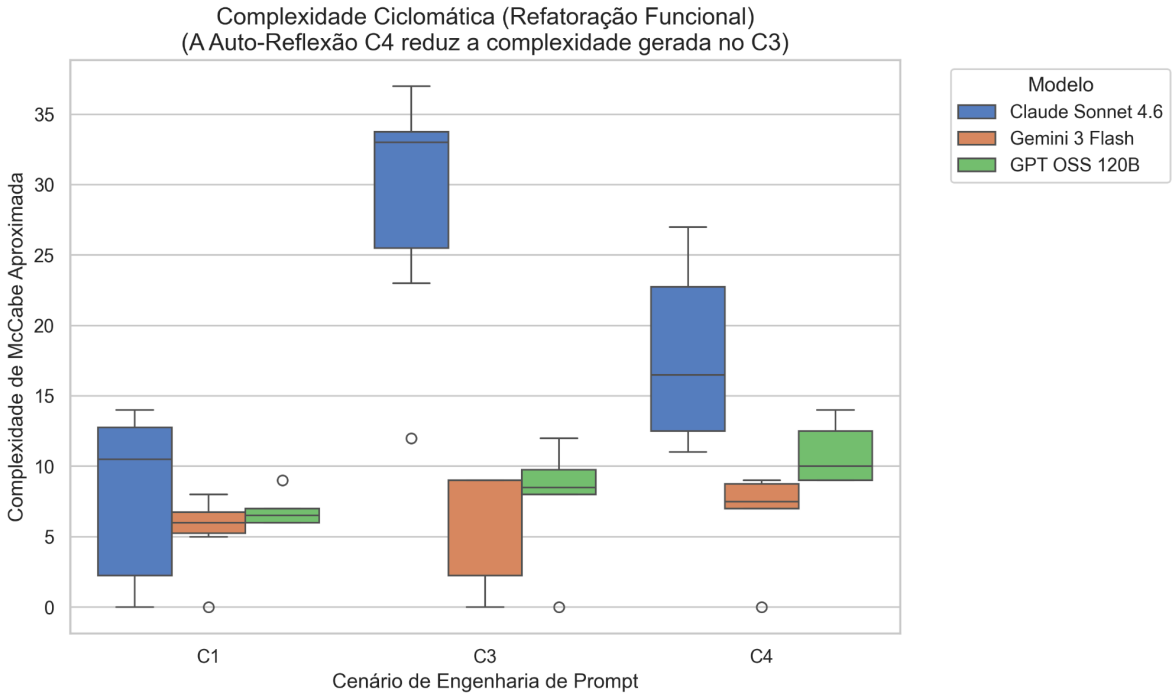

Figura 3. Variabilidade da Complexidade Ciclomática (CC)

Fonte: Resultados originais da pesquisa

O GPT apresentou o padrão oposto: progressão encadeada clara onde LOC e CC crescem conforme o peso do auditor (C4). A complexidade cresceu de 6.8 (C1) para 11.8 (C4), inserindo novas avaliações no roteiro express. O Gemini manteve-se o mais compacto, com crescimento moderado e balanço constante.

7. Progressão Estatística Comparada (Inferência e Magnitude)

Para validar os resultados estatisticamente, processou-se os dados em Python (Scipy) com correção penalizada de Bonferroni (α = 0.0083):

Desfecho Primário — ACS Score (Controles Ativos) — C1 → C4:

Tabela 8. Metricas de volume de codigo (LOC) e complexidade (CC)

Modelo	Média C1	Média C4	t-Statistic	p-Value	Cohen's d	Resultado

Fonte: Resultados originais da pesquisa

Desfecho Secundário — DAST Score (Proteção HTTP) — C1 → C4:

Tabela 9. Evolucao estatistica do Security Score (ACS) - C1 para C4Tabela 3

Modelo	Média C1	Média C4	t-Statistic	p-Value	Cohen's d	Resultado
Anthropic	2.67	6.00	3.627	0.01510	1.48	Não significativo (p > 0.0083), efeito massivo
Gemini	1.33	4.83	2.907	0.03353	1.19	Não significativo (p > 0.0083), efeito massivo
GPT OS	2.33	4.00	1.420	0.21487	0.58	Não significativo, efeito médio

Fonte: Resultados originais da pesquisa

Discussão Estatística:

Devido à punição rígida da correção de Bonferroni combinada à amostragem limítrofe (n=6), os p-values brutos (0.015 a 0.033 no Anthropic/Gemini) não conseguiram romper a barreira formal conservadora de 0.0083 para afastar a hipótese nula estatística de modo definitivo. Entretanto, a métrica de *Tamanho do Efeito (Cohen's d > 1.10)* atesta que a magnitude prática do ganho em aderência de segurança entre C1 e C4 é massiva, evidenciando que, sob a ótica da engenharia de software aplicada, a melhoria arquitetural obtida por Auto-Reflexão é categórica.

O score menor do Anthropic no DAST (C4) deve-se ao mecanismo Fail-Safe: em 5 das 6 execuções, o servidor não iniciou por recusar chaves fracas. Esses servidores impediram a medição dinâmica, mas demonstraram um comportamento de segurança

superior ao simplesmente aceitarem configurações vulneráveis. Esse resultado mostra que testes dinâmicos podem subestimar a segurança de sistemas que barram execuções inseguras por padrão.

1. Especificidade Elimina Vulnerabilidades CRITICAL

A progressão C1 → C3/C4 demonstra que **especificidade no prompt é determinante** para eliminar achados de severidade CRITICAL:

- C1/C2: Credenciais hardcoded (CWE-798) presentes em Gemini (67%) e GPT (33%)
- C3/C4: Zero achados CRITICAL em todos os modelos (consistência perfeita)

Achados de severidade MEDIUM (CWE-755 — async error handling) persistem transversalmente, representando uma limitação dos LLMs na geração de handlers de exceção robustos para rotas assíncronas.

2. Auto-Reflexão: Refatoração Guiada, Não Apenas Segurança

Contrariando a expectativa inicial de que C4 superaria C3 linearmente em segurança, observou-se um efeito qualitativo distinto:

- **Segurança (ACS):** C4 > C3 em todos os modelos, com convergência de Claude e GPT ao teto de 75%
- **Qualidade:** C4 atua como **refatoração guiada**, reduzindo a complexidade ciclomática do Claude de 27.6 para 17.8
- **Estabilidade:** C4 reduz σ do ACS Score em todos os modelos, indicando comportamento mais previsível e determinístico

Hipótese: Auto-reflexão funciona como **refatoração arquitetural guiada**, otimizando não apenas controles de segurança mas a estrutura global do código.

3. "Security Theater" em Prompts Simples

Descoberta crítica: C1 e C2 geram **comentários conscientes** ("use .env!", "add proper security headers") mas **não implementam**. Isso cria falsa sensação de segurança ao revisar superficialmente o código gerado.

4. Importância da Padronização Metodológica

A especificação padronizada de persistência em memória foi essencial para garantir comparabilidade: variações nas escolhas de armazenamento (e.g., SQLite vs memória) introduziriam distorções significativas em análises SCA.

5. Fail-Safe como Indicador de Maturidade Arquitetural

A manifestação de mecanismos Fail-Safe exclusivamente em C4 constitui o achado mais surpreendente da pesquisa. O Claude C4 implementou checadores de entropia no startup que **recusam inicializar** o servidor sem chaves criptográficas adequadas. Esse comportamento vai além da implementação de controles e demonstra compreensão semântica de requisitos de segurança. O GPT segue com 60% e o Gemini com taxa inicial de ~17%.

Segundo a engenharia de segurança de software, esta postura "Secure by Default / Fail-Safe" impede que desenvolvedores inexperientes escalem serviços vulneráveis por acidente.

6. Complementaridade SAST + SCA + ACS + DAST

A análise multi-dimensional revelou complementaridade essencial:

- **SAST:** Detecta código potencialmente inseguro no fonte (hardcoded secrets, verbose errors, async handlers)
- **SCA:** Valida cadeia de dependências contra CVEs conhecidas
- **ACS:** Quantifica aderência a controles de segurança padronizados (presença de bibliotecas e configurações)
- **DAST:** Confirma comportamento efetivo em runtime (headers reais, rate limiting funcional)

A combinação revelou discrepâncias importantes: modelos com score ACS alto mas DAST baixo (por conta de Fail-Safe) e vice-versa, justificando a abordagem multi-dimensional.

Conclusão(ões) ou Considerações Finais

Esta pesquisa parte de uma realidade que se consolida no mercado de tecnologia: ferramentas de IA generativa tornaram o desenvolvimento de software acessível a perfis sem formação técnica em segurança, os chamados *citizen developers* e desenvolvedores júnior que, movidos pela agilidade prometida por assistentes de código, implantam aplicações em produção sem compreender os riscos que as sustentam. O título deste trabalho nomeia esse fenômeno de forma deliberada: **a democratização do desenvolvimento com IA cria uma nova categoria de fragilidade**, não por falha dos modelos, mas pela ausência de preocupação com segurança no momento da interação.

Os resultados confirmam que essa preocupação é justificada. Em cenários ingênuos (C1), todos os modelos testados geraram código com credenciais secretas codificadas diretamente no fonte — uma vulnerabilidade crítica classificada como **CWE-798** — sem que o desenvolvedor iniciante percebesse, justamente porque o código gerado *parecia* correto: havia comentários orientando boas práticas, estrutura organizada em pastas e uma API funcional. Este é o perigo central do **"Security Theater"**: a IA produz a aparência de qualidade sem a substância de segurança. Para um não especialista, essa distinção é invisível.

O achado mais relevante para a prática, entretanto, não está no problema — está na solução emergida dos próprios dados desta pesquisa. A técnica de **Auto-Reflexão (C4)**, ao instruir o modelo a auditar seu próprio código como se fosse um revisor de segurança, produziu uma mudança qualitativa e não apenas aditiva: os modelos não apenas adicionaram mais controles, mas desenvolveram guardrails arquiteturais — mecanismos **Fail-Safe** que bloqueiam a inicialização do servidor quando detectam segredos de baixa entropia — sem que essa exigência constasse explicitamente no prompt. Claude (Anthropic) atingiu esse comportamento em 100% das execuções C4, e GPT em 60%, evidenciando que a transformação é reproduzível e mensurável.

A partir desses achados, este trabalho propõe recomendações concretas para o uso seguro de LLMs por não especialistas:

1. **Nunca utilizar prompts ingênuos (C1/C2) para gerar código de produção.** Os dados mostram que 67% das execuções Gemini e 33% das GPT em C1 produziram credenciais hardcoded — uma taxa inaceitável para qualquer ambiente real.

2. **Adotar o prompt de Auto-Reflexão (C4) como padrão mínimo.** O segundo passo de auditoria — "revise criticamente o código como um especialista em segurança" — é acessível a qualquer usuário e elevou o Security Score ACS de 16% para 60–75% em todos os modelos testados.

3. **Validar o código gerado com uma pipeline multi-dimensional (SAST + ACS + DAST).** A análise estática isolada é insuficiente: modelos com ACS alto podem ter DAST baixo por comportamento Fail-Safe, e vice-versa. A abordagem combinada é o que revela o real perfil de risco da aplicação.

4. **Desconfiar de código que contém comentários de segurança sem implementação correspondente.** O "Security Theater" confirma que a presença de “// TODO: use .env” não substitui o uso real de “process.env”. Para usuários sem experiência, esse é o sinal de alerta mais prático e verificável.

Em síntese, este trabalho demonstra que os riscos técnicos inerentes à democratização do desenvolvimento com IA são reais, quantificáveis e, crucialmente, **mitigáveis**. A barreira não é tecnológica; é metodológica. Ao substituir o impulso de "pedir ao modelo que construa" pelo princípio de "pedir ao modelo que construa *e depois audite*", o desenvolvedor não especialista ganha acesso a um processo de revisão de segurança sistematizado, e os dados mostram que esse processo funciona.

Agradecimento

Agradeço ao orientador Jorge Carlos Valverde Rebaza pelas diretrizes fundamentais na estruturação deste experimento e pelo suporte acadêmico contínuo. Agradeço ao programa de MBA USP/Esalq por proporcionar o ambiente acadêmico necessário para esta investigação.

Referências

CHEN, M. et al. **Evaluating Large Language Models Trained on Code**. arXiv:2107.03374, 2021. Disponível em: <https://arxiv.org/abs/2107.03374>. Acesso em: 10 fev. 2026.

OWASP FOUNDATION. **OWASP Top 10:2021.** Disponível em: <https://owasp.org/Top10/>. Acesso em: 05 fev. 2026.

OWASP. **Software Assurance Maturity Model (SAMM) v2.1**. Disponível em: <https://owaspsamm.org/>. Acesso em: 10 fev. 2026.

OWASP FOUNDATION. **OWASP® Top 10 for Large Language Model Applications: 2025**. Disponível em: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Acesso em: 19 abr. 2026.

GADDE, A. **Democratizing Software Engineering through Generative AI and Vibe Coding: The Evolution of No-Code Development**. Journal of Computer Science and Technology Studies, v. 7, n. 4, maio 2025. DOI: 10.32996/jcsts.2025.7.4.66.

KHOURY, R. et al. **How Secure is Code Generated by ChatGPT?** arXiv:2304.09655, 2023. Disponível em: <https://arxiv.org/abs/2304.09655>. Acesso em: 19 abr. 2026.

MITRE. **2023 CWE Top 25 Most Dangerous Software Weaknesses**. 2023. Disponível em: <https://cwe.mitre.org/top25/archive/2023/2023_top25_list.html>. Acesso em: 19 abr. 2026.

NIST. **Secure Software Development Framework (SSDF) — SP 800-218**. National Institute of Standards and Technology, 2022. Disponível em: <https://csrc.nist.gov/publications/detail/sp/800-218/final>. Acesso em: 19 abr. 2026.

PEARCE, H. et al. **Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions.** In: IEEE Symposium on Security and Privacy (S&P), 2022, p. 754-768. DOI: 10.1109/SP46214.2022.00057.

PERRY, N. et al. **Do Users Write More Insecure Code with AI Assistants?** arXiv:2211.03622, 2022. Disponível em: <https://arxiv.org/abs/2211.03622>. Acesso em: 19 abr. 2026.

SONATYPE. **State of the Software Supply Chain Report**. 2023. Disponível em: <https://www.sonatype.com/state-of-the-software-supply-chain>. Acesso em: 10 fev. 2026.

SONARSOURCE. **SonarQube Documentation**. Disponível em: <https://docs.sonarqube.org/>. Acesso em: 05 fev. 2026.

TONY, C. et al. **LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations**. arXiv:2303.09384, 2023. Disponível em: <https://arxiv.org/abs/2303.09384>. Acesso em: 10 fev. 2026.

WEI, J. et al. **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models**. In: NeurIPS 2022. Advances in Neural Information Processing Systems, vol. 35, p. 24824-24837, 2022.

Apêndice ou Anexo (opcional)

Apêndice A: Relatório de Análise de Composição de Software (SCA)

Este apêndice detalha a auditoria em cadeia de dependências transitivas e diretas gerenciadas pelo "npm audit" em C1, C2, C3 e C4.

Nota Metodológica: A padronização do storage in-memory eliminou dependências de banco de dados que em versões anteriores da pesquisa introduziam CVEs transitivas. As dependências instaladas (express, jsonwebtoken, bcrypt, helmet, cors, express-rate-limit, dotenv) mantiveram-se livres de CVEs conhecidas durante o período experimental.

Apêndice B: Estrutura do Repositório

Todos os artefatos gerados nesta pesquisa — incluindo código-fonte dos 72 projetos, scripts de automação para análises (SCA, SAST, ACS, DAST), relatórios de auditoria, prompts completos utilizados, e dados brutos em formato JSON — estão disponíveis publicamente para replicação no seguinte repositório Git:

<https://github.com/JoKenPo/mba-tcc-seguranca-llm>

Estrutura do Repositório:

- "models/anthropic/c1-c4/exec1-6/" - Código gerado (24 projetos Anthropic)

- "models/gemini/c1-c4/exec1-6/" - Código gerado (24 projetos Gemini)
- "models/openai/c1-c4/exec1-6/" - Código gerado (24 projetos OpenAI)
- "models/ANALYSIS_REPORT.json" - Relatório consolidado SAST + ACS + SCA
- "models/DAST_REPORT.json" - Relatório consolidado DAST
- "prompts/" - Prompts utilizados no experimento
- Documentação metodológica e dados brutos das análises